

Your robot. Your world. Your first simulation.

Getting started with **PyRobot Studio**

1

Choose a model

2

Set up the robot

3

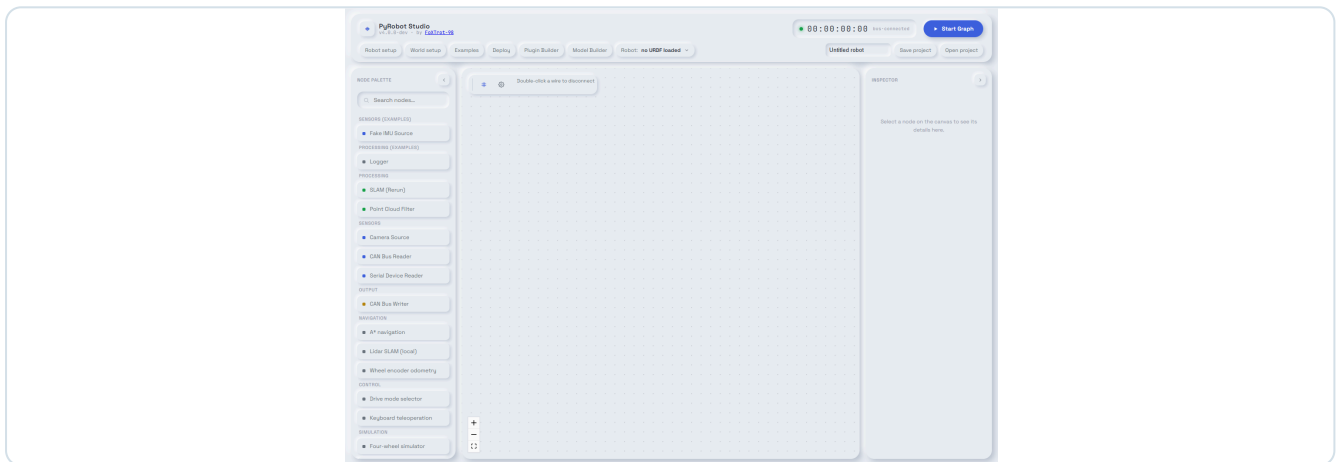
Check the world

4

Drive & explore

5

Save your project



Start with the top-bar setup buttons. You do not need to wire a graph by hand.

What you will make

A saved project containing your robot, a connected simulation graph, and the settings needed to reopen it. Start with the sample robot, then follow the same workflow for your own supported model.

[Install · 2](#) [Choose model · 3](#) [Own CAD · 4](#) [World · 7](#) [Drive · 9](#) [Save & help · 11](#)

Created by Kanishka Kularathna · FoXTrot-98

Project-owned guide and screenshots: Apache-2.0. Screenshots show the development UI; sample values are examples, not settings for every robot.

Install once. Start two services.

This is a developer-run application, not a desktop installer. If Studio already opens, skip to page 3. The locally verified platform is Windows.

Install Git, 64-bit Python **3.14.6**, Node.js **24.18.0** and npm, matching this repository's baseline. Download or clone the project, then open PowerShell in its root folder.

Prepare the checkout

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.lock.txt
.\.venv\Scripts\python.exe -m pip check
cd frontend
npm.cmd ci
npm.cmd run build
cd ..
```

Terminal 1 — backend, from the project root

```
.\.venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
```

Terminal 2 — interface, from the project root

```
cd frontend
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

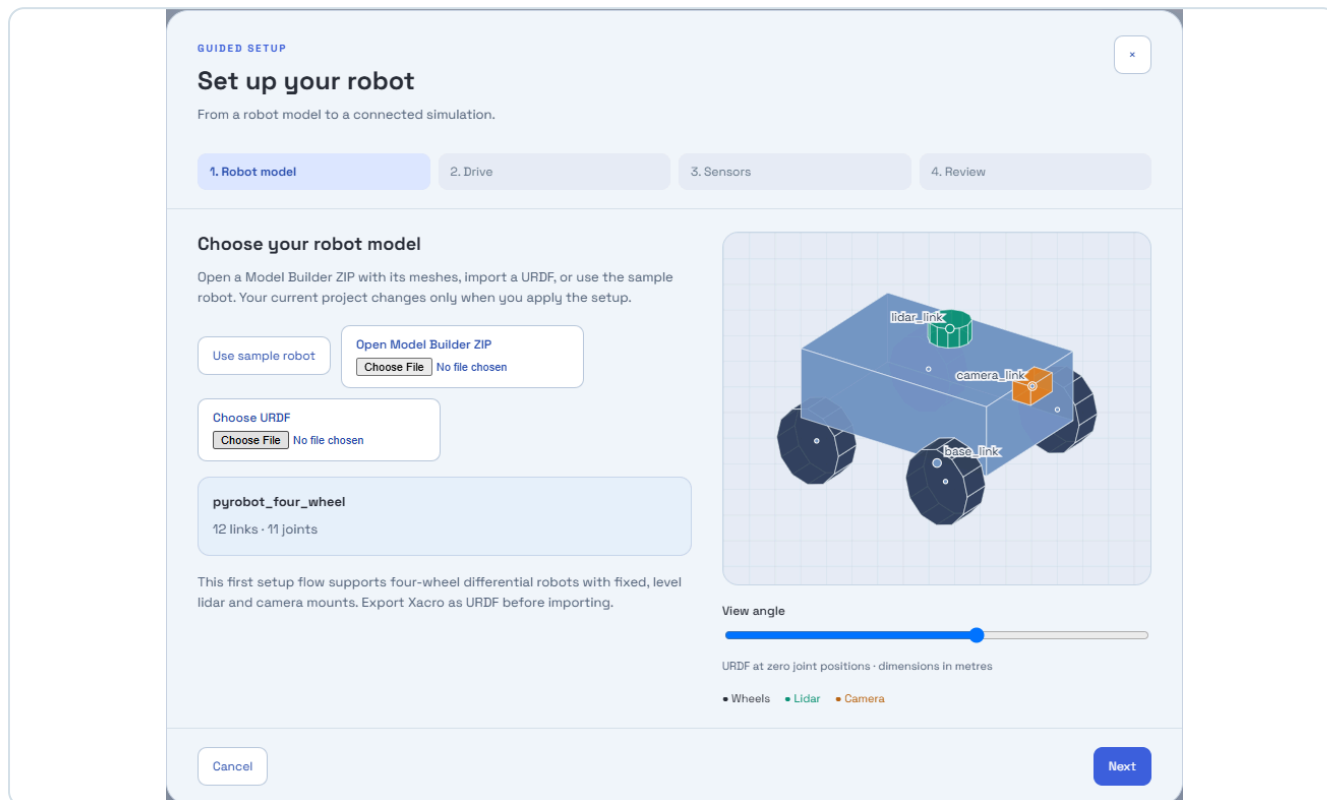
Open **<http://127.0.0.1:5173>** in your browser. Keep both terminals open. Use **Stop Graph** before shutting them down with Ctrl+C.

Webots is optional for your first run. Install a compatible Webots R2025 release to use Webots physics and external worlds. The built-in simulator needs no Webots installation.

Do not add unrestricted backend `--reload` when using Webots: generated controller files can trigger a backend restart. Detailed installation reference: `README.md` and `docs/BASELINE.md`.

Create a project through Robot setup.

On a fresh session, click **Robot setup**. If a project is already open, stop it and save it first. Applying a new simulation can replace the current graph.

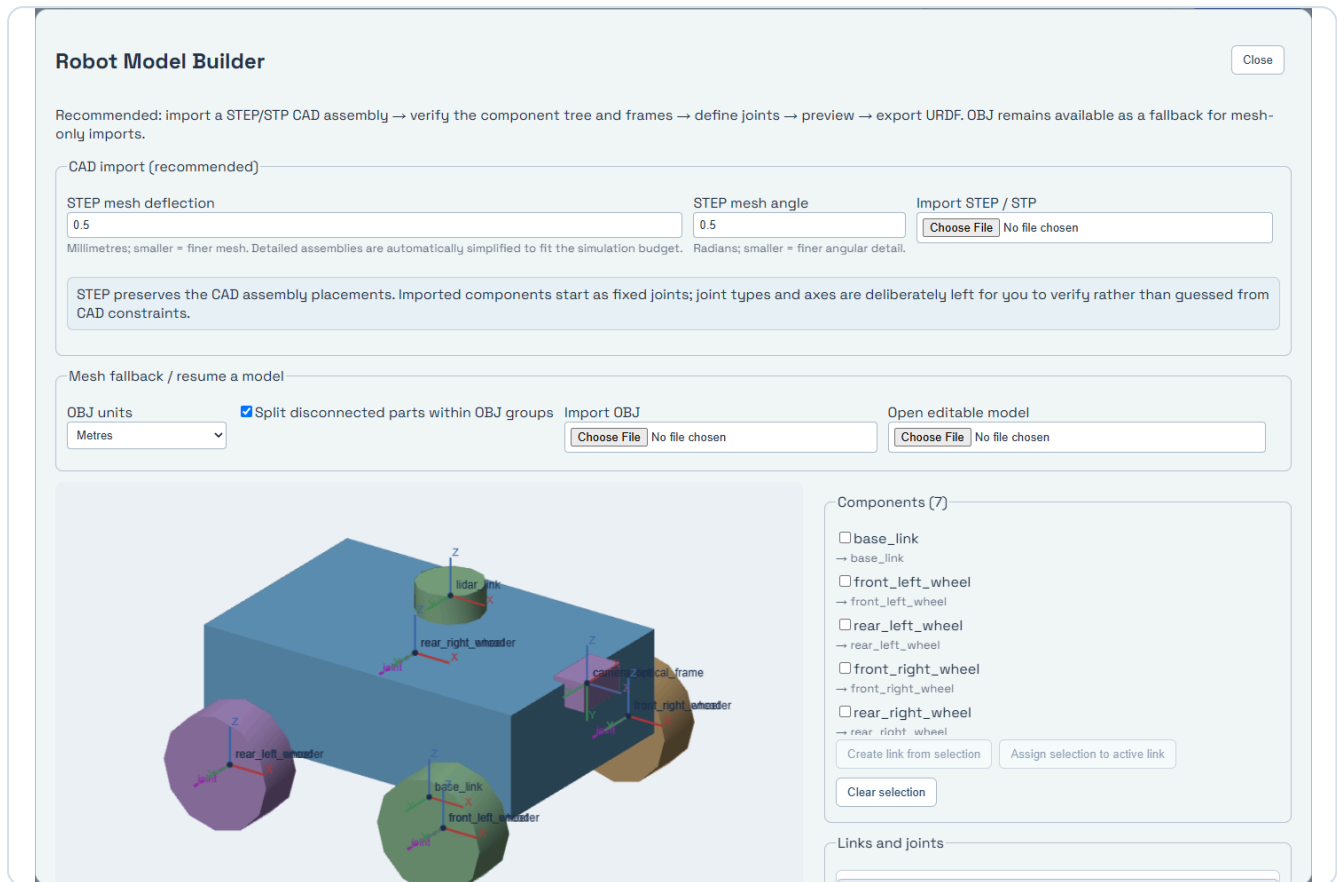


The model preview helps you check orientation and shape before applying anything.

You have...	Use this route
No robot file yet	Use sample robot , then Next. This is the quickest way to learn the controls.
A Studio builder ZIP	Open Model Builder ZIP . The bundle must contain its editable builder model.
A supported URDF	Choose URDF . A loose URDF does not automatically bundle its external mesh files.
A STEP/STP or OBJ model	Use Model Builder first; follow page 4.

Supported custom simulation: four-wheel differential drive, two equal-radius wheels per side, with fixed level lidar and camera frames. A CAD assembly alone does not define a working robot. Other drive types and arbitrary actuated mechanisms are not covered by this workflow.

Turn geometry into robot links.



Practice with `examples/model-builder/four-wheel-rover.robot-builder.json` using Open editable model.

1. Open **Model Builder**, then **Import STEP** or **Import OBJ**. Check dimensions against a known measurement. STEP imports use millimetres before conversion to metres.
2. Group rigid parts into links. Keep the chassis fixed; give each wheel a separate moving joint. Imported CAD placements begin as fixed joints.
3. Set the base at ground level with **+X forward, +Z up**. Put wheel centres one radius above ground. Wheel axes must point along base **+Y**.
4. Add fixed sensor frames for the lidar and camera. Set the correct offsets and level orientations. Confirm the scale/orientation checkbox.
5. Save an editable model for future changes. Click **Use in robot setup**, or **Export URDF bundle** and import that ZIP in Robot setup.

The direct handoff carries meshes, smooth normals and builder-assigned colors. No manual mesh copying is needed.

CAD textures/material finishes and measured dynamics are not imported. Simplify dense models: embedded assets are limited to 20,000 triangles and 6 MiB of compact asset JSON; builder ZIP upload is limited to 8 MiB. See `docs/MODEL_TO_SIMULATION.md`.

Check all four wheel assignments.

These are sample-robot values. Use measurements from your own robot.

1. Select the correct **Robot base frame**.
2. Assign distinct left-front, left-rear, right-front and right-rear wheel joints. Left/right are relative to the robot facing +X.
3. For mesh wheels, enter the measured **Wheel radius override (m)**. The included mesh rover uses 0.12 m; your robot may differ.
4. Set **Robot clearance radius** to enclose the body and wheels. **Planning clearance** must be at least as large.

Webots physics

Expand this section to enter body mass, mass per wheel, motor limits, damping, friction and slip. Body mass excludes the wheels.

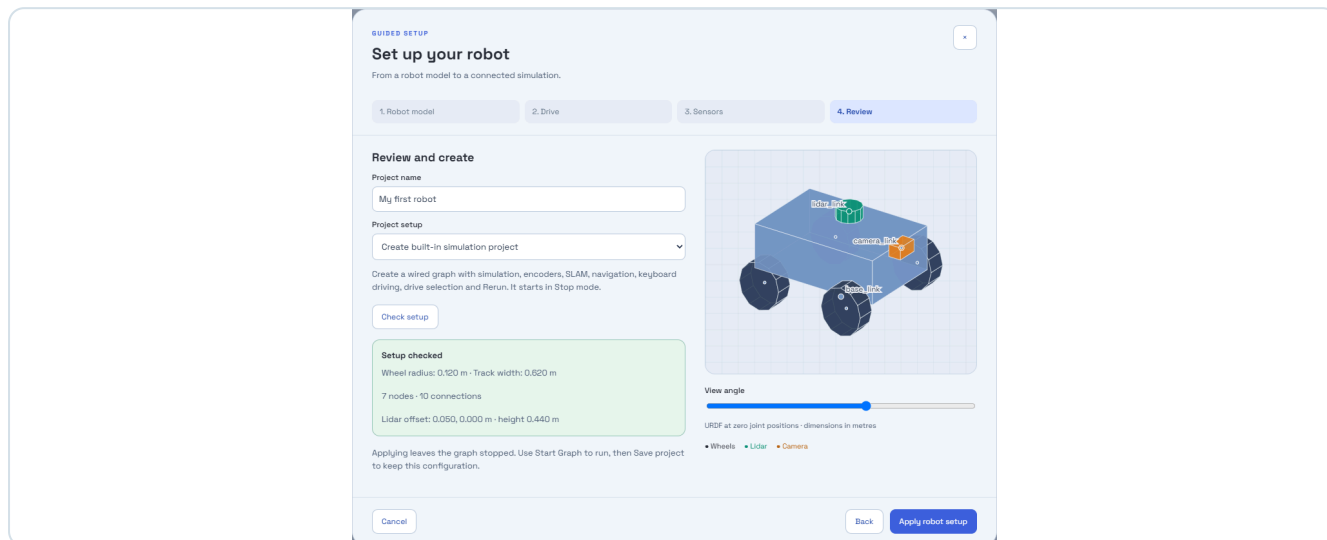
Check before continuing

Equal wheel radii. Four distinct joints. Correct pivots. All other joints fixed. Do not reduce clearance merely to bypass a validation error.

Physics settings are approximate and apply to Webots, not the built-in geometric simulator. They are not measured automatically from CAD.

Check setup, then apply.

In **Sensors**, choose the fixed lidar and camera mounting frames. Use the camera mount frame, not an optical frame with different axes. For the sample: `lidar_link` and `camera_link`.



Check setup validates the draft. Apply robot setup creates or updates the stopped project.

Project setup	When to choose it
Create built-in simulation project	First run in the generated room. Fast geometric simulation; no Webots required.
Create Webots simulation project	Webots physics or an external Webots world. Requires Webots installed.
Update robot in current graph	Keep an existing graph and replace its robot configuration.

1. Give the project a meaningful name.
2. Click **Check setup**. Resolve errors and review warnings.
3. If replacing a graph, save it first and acknowledge replacement.
4. Click **Apply robot setup**. Studio creates the connected simulation graph and leaves it stopped.

Click **Save project** after applying. For the built-in room, continue to page 9. For an external world, follow pages 7–8 first.

Select a world before driving.

World setup Close

Choose your environment before starting the robot.

Choose an installed Webots world or a local .wbt file. Its environment is preserved and supported existing robots are replaced with your robot.

Installed world

Select a world or enter a path below

External world path

D:/worlds/my-world.wbt

Path is on the computer running Studio's backend. Local assets must remain available.

Robot starting position

Use world coordinates in metres. Height is the robot base translation. Choose a clear, level floor; (0, 0) is not necessarily inside the room.

Check world opens a motor-disabled 3D preview. In Webots, select PyRobot four wheel and move it to clear floor, or edit its translation in the scene tree. Resume simulation to let it settle. Then use **Use Webots position** below and check again. You can also edit these coordinates directly.

Spawn X (m) Spawn Y (m)

Spawn heading (rad) Spawn height (m)

► Mapping area and resolution

Mapping bounds limit exploration; they do not supply obstacles. SLAM starts with unknown cells and uses lidar observations.

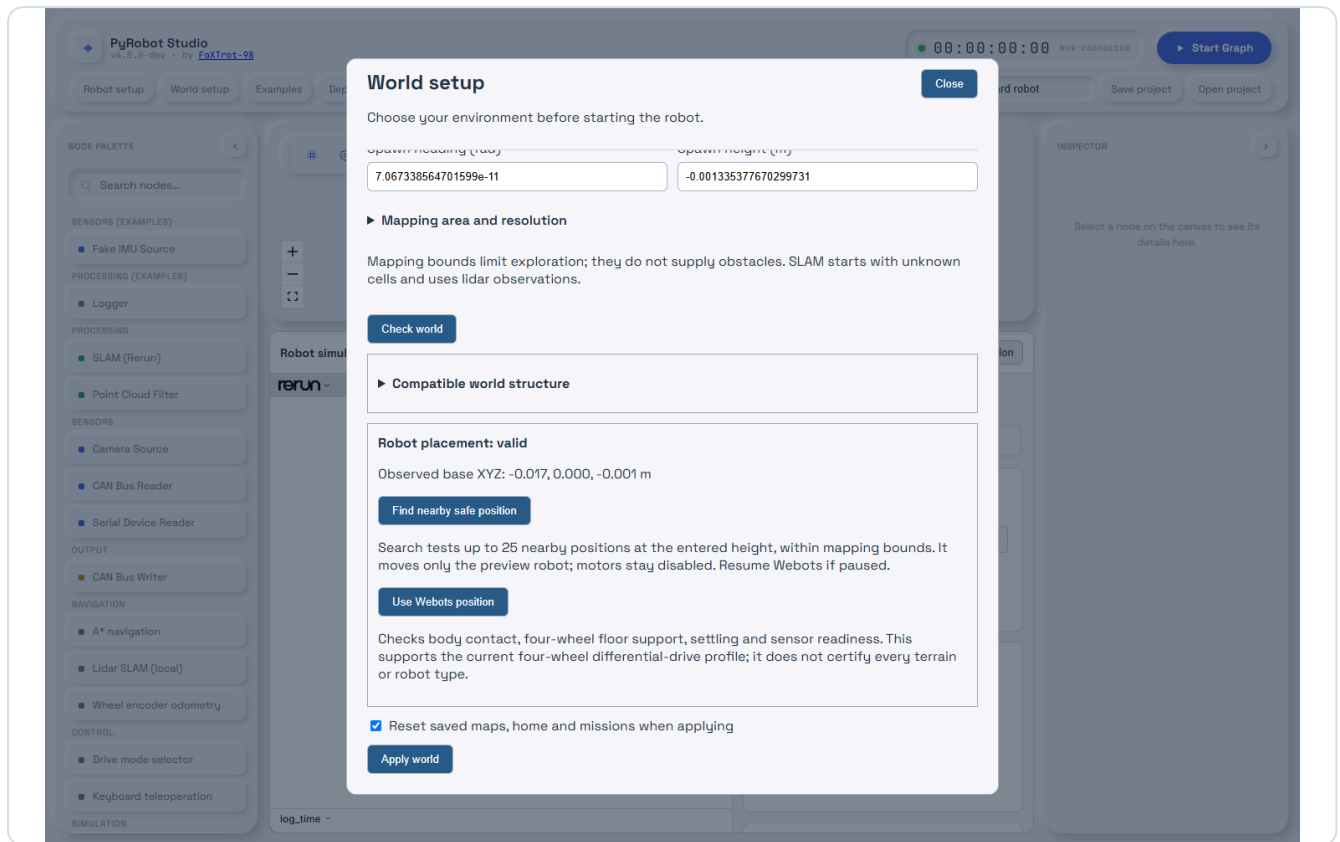
World selection is separate from Robot setup in the top bar.

1. Stop the graph and open **World setup**. Choose an installed world or enter an external .wbt path on the backend computer.
2. Set spawn X/Y, heading in radians, and base height. Choose open, level floor with enough room for the whole robot.
3. Review mapping bounds and resolution. Bounds limit the map; they do not supply an obstacle map or guarantee that floor exists there.
4. Click **Check world**. Read the compatibility results and inspect the placement preview in Webots.

An installed-world listing is not a compatibility guarantee. Supported imports use R2025 ENU (Z-up) worlds. Studio replaces existing top-level robots; review the removal list. Unsupported structures produce an explanation.

World setup can also be used before Robot setup. In that case, return here after configuring the robot to check its physical placement. Changing worlds resets saved maps, home positions and missions.

Four supported wheels. A clear body.



Actual placement-check screenshot. Coordinates shown belong to that test, not to your world.

1. Keep the preview simulation running so the robot can settle. Preview motors stay disabled.
2. If placement is invalid, move to clear floor in Webots or correct the numeric spawn fields. You can also try **Find nearby safe position**.
3. Inspect a suggested position. When available, click **Use Webots position** to adopt the observed pose.
4. Click **Check world** again. Wait for **Robot placement: valid**, acknowledge the map/mission reset, then **Apply world**.

The nearby search checks up to 25 positions at the entered height. It can be cancelled. It does not search every room or find another floor automatically.

A compatible world structure is only the first check. Physical support, body/wheel contact, settling and sensor readiness must also pass. Startup repeats these checks. A valid spawn does not qualify all terrain or prove a complete navigation route.

Drive a little. Check what you see.

1. Click **Start Graph**. Wait for sensor data and the measured map; Webots may take time to launch.
2. Select **Manual**. Click the **Keyboard driving** pad so it has focus.
3. Hold W briefly, then release. Check that the visible robot moves forward and stops.
4. Try a small turn. Check that the visual robot, estimated pose and measured walls behave consistently.
5. Use **Stop** to keep sensors running while stopping motion. Use **Stop Graph** to end the runtime.

W A S D

W / ↑ forward

S / ↓ reverse

A / ← and D / → turn

Space: zero velocity

If walls drift badly or the visual robot disagrees with its estimated pose, stop and check wheel assignment, radius, scale, sensor frames and slip before trying autonomy.

The screenshot shows the Webots control interface. At the top, there are three buttons: **Manual** (highlighted in blue), **Autonomous**, and **Stop**. Below these is a **Keyboard driving** section with instructions: "Click here, then hold W/A/S/D or arrow keys" and "Releasing keys or leaving this pad stops manual motion." Underneath are three tabs: **Navigate**, **Maps**, and **Settings**. The **Navigate** tab is active, showing a **Mission** section with input fields for **Goal X** (0) and **Y** (0), a **Navigate** button, and a "goal: paused" status with a **Cancel mission** button. At the bottom is a **Home position** section showing "Home: 0.00, 0.00 m" with **Set home here** and **Return home** buttons. A note below says: "Set home here pauses navigation and saves the position with the project. Use it only in the same map."

Controls can scroll. Rerun is the 3D view; the adjacent SLAM map is used for waypoint selection.

What the map means

Black cells are occupied; gray cells are unknown; light cells are measured free space. Mapping starts from sensor observations, not from a perfect copy of the Webots world.

Start with one short mission.

1 · Set home

The first map pose becomes home by default. At a suitable known position, **Set home here** saves a new home and pauses navigation.

2 · Try a waypoint

On the **SLAM waypoints** map, click a nearby clear destination, then **Run waypoints**. Draft clicks alone do not start motion.

3 · Explore

Explore map selects reachable frontiers. It returns after the configured target limit or when no eligible frontiers remain.

4 · Return

Return home requests a route back and alignment to the home heading. **Home reached** means stopped at the estimated home; it is not docking.

Keep the mission under control

Control / status	Meaning
Pause motion / Resume motion	Pause or resume the current autonomous mission.
Cancel mission	Stop that mission. Choose a new mission to move again.
no_path / obstacle_stop	Wait stopped while fresh map data and replanning may clear the blockage.
navigation_failed / stalled	Inspect the robot and route. Clear the cause, then use Retry navigation or choose a new goal.
sensor_timeout	Motion is stopped while waiting for fresh mapping data.

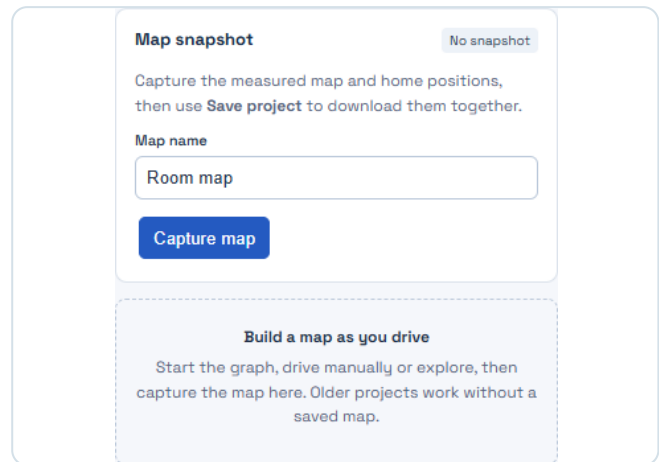
Choose algorithms after the first successful run

In **Settings**, the available planners are A* and Dijkstra; controllers are proportional and fuzzy logic. Begin with the defaults. Changing algorithms does not fix a wrong wheel radius, unsupported placement or drifting localization.

Exploration can stop before a whole building is mapped. Global localization, loop closure and arbitrary-world reliability are not established. Test your own environment in small stages.

Keep a project you can reproduce.

1. Use **Save project** to download the applied robot, graph and settings.
2. To keep a measured map, open **Maps**, enter a name and click **Capture map**, then save the project again.
3. Later, use **Open project** to restore the saved JSON. Confirm the starting pose when using a saved map.
4. Keep the editable builder model too. Keep external Webots worlds and referenced assets at accessible paths.



A project embeds builder geometry and captured map data, but not arbitrary world assets or installed plugins.

Problem	First thing to check
Setup button disabled	Stop Graph and wait for the current operation to finish.
Duplicate wheels / unsupported model	Four distinct moving wheel joints, equal radii, base +Y axes and fixed sensor mounts.
Placement stays invalid	Resume Webots if paused; inspect body contact and all four wheels. Adopt a safe observed pose, then recheck.
Robot will not drive	Wait for sensor readiness, select Manual and focus the keyboard pad.
No route / repeated stops	Choose a nearby open goal; keep configured clearance. Map more space manually if needed.
Changes disappeared	Apply the setup first, then Save project. Unsaved drafts are not the project.

Your first-project checklist

- ✓ Model scale and frames checked
- ✓ Wheel assignments checked
- ✓ World placement valid, if used
- ✓ Short manual drive passed
- ✓ Short mission and return tested
- ✓ Project saved and reopened

More detail in the repository: `docs/GUIDED_ROBOT_SETUP.md`, `docs/MODEL_TO_SIMULATION.md`, `docs/SIMULATION_WORLDS.md` and `docs/AUTONOMOUS_NAVIGATION.md`. Report problems using `CONTRIBUTING.md`; include world, robot, steps and logs.